

E E P R O M / I A P 功能

S T C 8 9 C 5 2 R C 内置 4 K B E E P R O M，地址范围 2 0 0 0 H - 2 F F F H，共 8 个扇区，每扇区 5 1 2 B。E E P R O M 与程序空间独立，可在用户程序运行时通过 I A P 进行读写操作。擦除以扇区为单位，写入以字节为单位。F l a s h 擦写寿命 1 0 万次以上。

E E P R O M 空间分布

扇区	起始地址	结束地址	大小
0	2 0 0 0 H	2 1 F F H	5 1 2 B
1	2 2 0 0 H	2 3 F F H	5 1 2 B
2	2 4 0 0 H	2 5 F F H	5 1 2 B
3	2 6 0 0 H	2 7 F F H	5 1 2 B
4	2 8 0 0 H	2 9 F F H	5 1 2 B
5	2 A 0 0 H	2 B F F H	5 1 2 B
6	2 C 0 0 H	2 D F F H	5 1 2 B
7	2 E 0 0 H	2 F F F H	5 1 2 B

I A P 寄存器

I S P _ D A T A (E 2 H) : I A P 数据寄存器。
I S P _ A D D R H (E 3 H) : I A P 地址寄存器高字节。
I S P _ A D D R L (E 4 H) : I A P 地址寄存器低字节。
I S P _ C M D (E 5 H) : I A P 命令寄存器。
 B 1 B 0 = 0 1 : 字节读
 B 1 B 0 = 1 0 : 字节写
 B 1 B 0 = 1 1 : 扇区擦除
I S P _ T R I G (E 6 H) : I A P 触发寄存器，依次写入 0 x 4 6 和 0 x B 9 触发操作。
I S P _ C O N T R (E 7 H) : I A P 控制寄存器，B 7 位 I S P E N 为使能位。

EEPROM 操作步骤

字节读操作：

1. 设置 `ISP_CONTR` 使能 `IAP` 功能
2. 设置 `ISP_CMD = 0x01` (字节读命令)
3. 设置 `ISP_ADDRH` 和 `ISP_ADDRL` 为目标地址
4. 向 `ISP_TRIG` 依次写入 `0x46` 和 `0xB9`
5. 从 `ISP_DATA` 读取数据

字节写操作：

1. 设置 `ISP_CONTR` 使能 `IAP`
2. 设置 `ISP_CMD = 0x02` (字节写命令)
3. 设置目标地址
4. 将待写数据写入 `ISP_DATA`
5. 触发 `IAP` 操作

扇区擦除操作：

1. 设置 `ISP_CMD = 0x03` (扇区擦除命令)
2. 设置扇区内任意地址
3. 触发 `IAP` 操作

注意：擦除会将整个扇区 512 字节全部置为 `0xFF`

EEPROM 编程示例

```
unsigned char EEPROM_Read(unsigned int addr) {  
    ISP_CONTR = 0x83;  
    ISP_CMD = 0x01;  
    ISP_ADDRH = addr >> 8;  
    ISP_ADDRL = addr & 0xFF;  
    ISP_TRIG = 0x46;  
    ISP_TRIG = 0xB9;  
    return ISP_DATA;  
}
```

```
void EEPROM_Write(unsigned int addr, unsigned char dat){  
    ISP_CONTR = 0x83;  
    ISP_CMD = 0x02;  
    ISP_ADDRH = addr >> 8;  
    ISP_ADDRL = addr & 0xFF;  
    ISP_DATA = dat;  
    ISP_TRIG = 0x46;  
    ISP_TRIG = 0xB9;  
}
```

```
void EEPROM_Erase(unsigned int sector_addr) {  
    ISP_CONTR = 0x83;  
    ISP_CMD = 0x03;  
    ISP_ADDRH = sector_addr >> 8;  
    ISP_ADDRL = sector_addr & 0xFF;  
    ISP_TRIG = 0x46;  
    ISP_TRIG = 0xB9;  
}
```